

Bilan de travail – FPSU avec BICYCL

Yasser NAMATY
Laboratoire Jean Kuntzmann (LJK)

2026

Travail réalisé

1. Intégration de BICYCL dans volePSI

Mise en place du chiffrement homomorphe additif CL HSM_{qk} (BICYCL) au sein de volePSI : implémentation de KeyGen, Encrypt, Decrypt et Add homomorphe, et développement du serveur FPSUServer_bicycl. Ajout et exécution des tests `test_CL_HSMqk_128` pour valider le bon fonctionnement du schéma.

2. Validation du protocole OKVS + chiffrement

Tests de décodage réalisés sur un ensemble $n = 8$, $m = 5$:

- clés synchronisées entre les deux serveurs → décodage réussi ;
- clés désynchronisées ou aléatoires → échec correctement détecté, sans fausse acceptation.

Cela confirme que le décodage de l'OKVS et la reconstruction homomorphe fonctionnent comme prévu.

3. Benchmarks BICYCL vs Paillier

Comparaison à niveau de sécurité équivalent :

Critère	BICYCL	Paillier	Vainqueur
Taille des chiffrés	4 663 bits	6 144 bits	BICYCL
Génération de clés	24.45 ms	183–1353 ms	BICYCL
Chiffrement	9.44 ms	85–164 ms	BICYCL
Déchiffrement	23.58 ms	3.7–22 ms	Paillier
Addition homomorphe	11 073 μ s	70 μ s	Paillier

BICYCL produit des chiffrés 24% plus compacts et un chiffrement jusqu'à $17\times$ plus rapide, mais l'addition homomorphe est environ $158\times$ plus lente que Paillier. BICYCL est donc préférable quand la compacité et le coût de chiffrement dominant (communication, nombreux clients), tandis que Paillier reste préférable pour les protocoles effectuant de nombreuses additions homomorphes.

4. Mise en place de l'intégration PIR / SimplePIR

Développement d'un pont Go/C++ (`bridge.go`) exposant les primitives SimplePIR (`PIRServerSetup`, `PIRClientQuery`, `PIRServerAnswer`, `PIRClientExtract`) sous forme de bibliothèque partagée (`libsimplepir.so`) compilée et liée à volePSI (`pir_demo`, `FPSUServer_PIR`, `FPSUServer_p`). Objectif : interroger uniquement les positions nécessaires du `storage` de l'OKVS (partie `sparse`) sans révéler la position recherchée, au lieu de transférer l'ensemble de la structure.

Avancement de la semaine

Le **SparseDecode1 avec PIR** a été terminé cette semaine. Sur cette lancée, je me suis lancé directement sur le **SparseDecode32 avec PIR**, sans repasser par la version sans PIR au préalable. Je n'ai pas encore réussi à le finaliser, mais j'ai bien compris la méthode à suivre pour implémenter

le PIR dans le cas du SparseDecode32, ce qui devrait permettre de terminer l'implémentation prochainement.

Prochaines étapes

- Finaliser **SparseDecode32 avec PIR** ;
- Intégrer la version finale qui inclut le chiffrement **BICYCL** et le **PIR** ;
- Passer à l'échelle sur des ensembles de taille réaliste ;
- Envisager un chiffrement/déchiffrement par lots (*batch*) pour réduire le coût de BICYCL sur les additions homomorphes.